DOCUMENTO TÉCNICO DE ENGENHARIA DE SOFTWARE & SEGURANÇA

Relatório de Auditoria de Segurança & Guia de Resolução em Grupo

Avaliação Técnica Exaustiva das 5 Falhas Críticas & Especificação Passo a Passo de Remediação para a Equipe

---

Projeto:	PsicoConnect — Plataforma Integrada de Atendimento Psicológico (v2.0.0 Monorepo)
Data de Realização:	16 de Setembro de 2026
Escopo Auditado:	backend/ (src/server.js, routes/, middleware/, config/), database/ (schema.sql, seed.sql), frontend/ (HTMLs, js/script.js), CI/CD (.github/workflows/ci.yml)
Finalidade:	Comprovação de Atividade Prática de Segurança da Informação e Manual de Remediação em Grupo
Classificação:	**CONFIDENCIAL — RESTRIÇÃO DE ACESSO TÉCNICO**

Visão Geral Metodológica & Stack Detectada:

A base de código do **PsicoConnect** foi inspecionada de forma sistemática e abrangente. Foi detectada a seguinte stack técnica:
• **Linguagem & Runtime:** JavaScript / Node.js (ES Modules, Node 20+).
• **Backend Framework:** Express 4.21.2 com middlewares de rate-limiting, helmet e cors.
• **Banco de Dados & Driver:** PostgreSQL nativo via driver pg (Pool connection), raw SQL parametrizado sem ORM.
• **Autenticação & Tokens:** JSON Web Tokens (via jsonwebtoken) e bcryptjs para hash de senhas.
• **Frontend:** Arquitetura híbrida (Vanilla HTML5 + JS DOM + CSS3) com persistência em localStorage e chamadas parciais REST.
• **CI/CD:** GitHub Actions com testes automatizados contra PostgreSQL em container de serviço.

Mapeamento das 5 Categorias:
1. *Banco sem Tranca:* PostgreSQL sem RLS em schema.sql, exigindo isolamento lógico por filtros WHERE usuario_id = $x no Express.
2. *Permissão no Navegador:* Validações de papel executadas apenas na interface web (ex: session()?.tipo === 'Administrador') sem restrição no backend.
3. *IDOR:* Handlers que consomem :id de objetos (agendamentos, mensagens) sem conferir pertinência do usuário.
4. *Chaves Expostas:* Segredos e chaves com fallbacks permissivos hardcoded em código e credenciais demo.
5. *Inputs sem Tratamento (XSS):* Manipulação direta de innerHTML sem bibliotecas de sanitização (como DOMPurify).

1. Lista Exaustiva de Achados de Segurança (Arquivo por Arquivo, Linha por Linha)

Sev.	Categoria	Arquivo e Linha	Descrição Sintética do Risco
CRÍTICA	3. IDOR	databaseRoutes.js:189-204	Cancelamento de consultas de terceiros por qualquer usuário autenticado.
CRÍTICA	4. Chaves Expostas	authMiddleware.js:4	Segredo JWT com fallback padrão hardcoded sem fail-fast no boot.
ALTA	1. Banco sem Tranca	databaseRoutes.js:207-221	GET /api/mensagens/:id sem verificar se o usuário é participante da consulta.
ALTA	1. Banco sem Tranca	databaseRoutes.js:224-241	POST /api/mensagens permite injeção de mensagens em consultas alheias.
ALTA	3. IDOR / Aberta	server.js:136-168	POST /api/meet/create-event público e sem autorização (spam e abuso de cota).
ALTA	5. XSS	script.js:247	Cross-Site Scripting armazenado no chat via concatenação direta em innerHTML.
MÉDIA	2. Permissão Navegador	script.js:155, 209-215	Painel admin restrito apenas por verificação de flag mock no localStorage.
MÉDIA	2. Permissão Navegador	databaseRoutes.js:78-108	POST /api/psicologos público permite injeção de perfil sem senha.
MÉDIA	4. Chaves / SSL	db.js:7	rejectUnauthorized: false desabilita validação de certificado SSL no PostgreSQL.
MÉDIA	5. XSS	script.js:238	DOM-based XSS na tela de triagem via innerHTML.
BAIXA	4. Chaves Expostas	seed.sql:11-15	Credenciais de demonstração (admin@psicoconnect.com / 123456) em seed.

Detalhamento dos Achados Críticos e Altos:

• Achado 1: IDOR no Cancelamento de Agendamentos (CRÍTICA)

Arquivo: backend/src/routes/databaseRoutes.js:189-204

```
router.patch('/agendamentos/:id/cancelar', requireAuth, async (req, res) => {
const { id } = req.params;
const result = await query(
`UPDATE agendamentos SET status = 'cancelado' WHERE id = $1 RETURNING *`,
[id]
);
```

Explorabilidade & Impacto: Qualquer paciente ou psicólogo autenticado na plataforma pode cancelar consultas de outros profissionais e pacientes arbitrariamente fornecendo o identificador UUID da consulta. Não há verificação de vínculo entre o usuário da sessão (req.user.id) e o paciente_id ou psicologo_id do agendamento.

• Achado 2: Segredo JWT com Fallback Público Hardcoded (CRÍTICA)

Arquivo: backend/src/middleware/authMiddleware.js:4

```
const JWT_SECRET = process.env.JWT_SECRET || 'psicoconnect_dev_secret_troque_em_producao';
```

Explorabilidade & Impacto: Se a variável JWT_SECRET for omitida no ambiente de deploy (Render, Docker, etc.), o sistema sobe em silêncio utilizando um segredo estático público. Qualquer atacante pode assinar tokens JWT forjados com { tipo: 'admin' } e obter acesso administrativo irrestrito à API.

• Achado 3: Vazamento de Mensagens Clínicas Confidenciais (ALTA)

Arquivo: backend/src/routes/databaseRoutes.js:207-221

```
router.get('/mensagens/:agendamentoId', requireAuth, async (req, res) => {
const result = await query(
`SELECT m.*, u.nome AS autor_nome FROM mensagens m ... WHERE m.agendamento_id = $1`,
[req.params.agendamentoId]
);
```

Explorabilidade & Impacto: Quebra grave de sigilo profissional médico-psicológico e violação da LGPD. O endpoint lista conversas privadas da consulta sem validar se o requisitante autenticado é o paciente ou o psicólogo responsável pela sessão.

• Achado 4: Rota de Criação do Google Meet Desprotegida (ALTA)

Arquivo: backend/src/server.js:136-168

```
app.post('/api/meet/create-event', async (req, res) => {
const { consultaId, summary, description, startDateTime, endDateTime, pacienteEmail, psicologoEmail } =
req.body;
```

Explorabilidade & Impacto: Endpoint totalmente público (sem requireAuth) que aceita consultaId e e-mails de destino arbitrários. Permite a atacantes esgotar cotas da API Google Calendar da clínica e enviar e-mails de phishing/spam legítimos a terceiros.

• Achado 5: Stored Cross-Site Scripting (XSS) no Chat (ALTA)

Arquivo: frontend/js/script.js:247

```
byId('chatMensagens').innerHTML = banco.mensagens.map(m =>
`<div class="chat-bubble ${m.autorTipo===s.tipo?'mine':''}"><b>${m.autorTipo}</b><p>${m.texto}</p></div>`
).join('');
```

Explorabilidade & Impacto: As mensagens enviadas no chat são injetadas diretamente via innerHTML sem sanitização ou escape. Cargas maliciosas executam scripts no contexto do navegador do psicólogo, permitindo roubo de sessões e manipulação de prontuários.

2. Pontos Fortes Identificados (Validados no Código Real)

✔ 1. Prevenção Efetiva contra SQL Injection (SQLi)

Evidência: backend/src/routes/databaseRoutes.js e authRoutes.js

Todas as operações de consulta e manipulação de dados utilizam exclusivamente queries parametrizadas com placeholders ($1, $2...) através do driver pg nativo. Não foi identificada nenhuma concatenação insegura de strings em comandos SQL.

✔ 2. Armazenamento Criptografado de Credenciais

Evidência: backend/src/routes/authRoutes.js:46, 121

As senhas de usuários nunca são armazenadas em texto simples. O sistema aplica o algoritmo bcrypt (bcrypt.hash(senha, 10)) no cadastro e realiza a comparação segura de hashes (bcrypt.compare) no fluxo de login.

✔ 3. Política Estruturada de Limitação de Requisições (Rate Limiting)

Evidência: backend/src/server.js:36-50

A API implementa express-rate-limit de forma em camadas: um limitador global (100 requisições / 15 min) e um limitador restritivo específico para as rotas de autenticação /api/auth (20 requisições / 15 min), mitigando ataques de força bruta e credential stuffing.

✔ 4. Isolamento Correto na Listagem Inicial de Agendamentos

Evidência: backend/src/routes/databaseRoutes.js:121-155

O endpoint GET /api/agendamentos inspeciona o token autenticado e aplica cláusulas WHERE rigorosas (WHERE ps.usuario_id = $1 para psicólogos e WHERE pa.usuario_id = $1 para pacientes), garantindo que a listagem inicial não vaze consultas alheias.

✔ 5. Controle de Acesso Baseado em Papéis (RBAC) em Rotas Críticas

Evidência: backend/src/routes/databaseRoutes.js:37, 55

As rotas de consulta geral e homologação de novos psicólogos (/api/psicologos/todos e /api/psicologos/:id/validacao) implementam o middleware encadeado requireAuth e requireRole('admin'), bloqueando tentativas de acesso de pacientes ou psicólogos comuns.

3. Guia Técnico de Resolução em Grupo (Plano de Remediação)

Para a execução colaborativa da remediação, foi estruturada uma matriz de divisão de tarefas distribuída entre 4 integrantes do grupo de desenvolvimento, acompanhada das correções de código completas:

Membro / Papel	Módulo Responsável	Vulnerabilidades Atribuídas	Arquivos Alvo
Dev 1 (Backend Auth)	Autenticação e Startup	• Segredo JWT hardcoded • Fail-fast no boot do servidor • Reativação da CSP no Helmet	authMiddleware.js server.js .env.example
Dev 2 (Backend Logic)	Regras de Negócio e APIs	• IDOR no cancelamento de consultas • Isolamento do chat (GET e POST) • Proteção da rota do Meet	databaseRoutes.js server.js
Dev 3 (Frontend DOM)	Interface e Sanitização	• Mitigação de XSS (Chat e Triagem) • Integração do Admin com a API real • Limpeza de credenciais demo	script.js admin.html login.html
Dev 4 (Database & QA)	Banco de Dados e Testes	• Saneamento do seed.sql • Correção do SSL rejectUnauthorized • Bateria de testes de regressão	schema.sql, seed.sql db.js database.test.js

Passo a Passo das Correções Técnicas (Antes vs. Depois):

Correção 1 (Dev 1): Fail-fast e Eliminação de Fallback no Segredo JWT

```
// backend/src/middleware/authMiddleware.js:4
- const JWT_SECRET = process.env.JWT_SECRET || 'psicoconnect_dev_secret_troque_em_producao';
+ const JWT_SECRET = process.env.JWT_SECRET;
+ if (!JWT_SECRET && process.env.NODE_ENV !== 'test') {
+ console.error('FATAL: A variável JWT_SECRET não está definida!');
+ process.exit(1);
+ }

// backend/src/server.js (logo após imports):
+ if (process.env.NODE_ENV === 'production' && (!process.env.JWT_SECRET || process.env.JWT_SECRET.length <
32)) {
+ console.error('FATAL: JWT_SECRET fraca ou ausente em produção!');
+ process.exit(1);
+ }
```

Correcão 2 (Dev 2): Verificacão de Propriedade contra IDOR no Cancelamento

```
// backend/src/routes/databaseRoutes.js:189-204
router.patch('/agendamentos/:id/cancelar', requireAuth, async (req, res) => {
const { id } = req.params;
+ const userId = req.user.id;
+ const userRole = req.user.tipo;
try {
+ let result;
+ if (userRole === 'admin') {
+ result = await query('UPDATE agendamentos SET status = \'cancelado\' WHERE id = $1 RETURNING *', [id]);
+ } else {
+ result = await query(`
+ UPDATE agendamentos SET status = 'cancelado' WHERE id = $1 AND (
+ paciente_id IN (SELECT id FROM pacientes WHERE usuario_id = $2)
+ OR psicologo_id IN (SELECT id FROM psicologos WHERE usuario_id = $2)
+ ) RETURNING *`, [id, userId]);
+ }
if (result.rows.length === 0) {
- return res.status(404).json({ ok: false, error: 'Agendamento não encontrado.' });
+ return res.status(403).json({ ok: false, error: 'Acesso negado ou consulta inexistente.' });
}
res.json({ ok: true, agendamento: result.rows[0], mensagem: 'Consulta cancelada.' });
```

Correção 3 (Dev 2): Isolamento de Participantes nas Mensagens do Chat

```
// backend/src/routes/databaseRoutes.js:207-241
+ async function usuarioParticipaDoAgendamento(agendamentoId, user) {
+ if (user.tipo === 'admin') return true;
+ const check = await query(`
+ SELECT a.id FROM agendamentos a
+ LEFT JOIN pacientes pa ON pa.id = a.paciente_id
+ LEFT JOIN psicologos ps ON ps.id = a.psicologo_id
+ WHERE a.id = $1 AND (pa.usuario_id = $2 OR ps.usuario_id = $2)`,
+ [agendamentoId, user.id]);
+ return check.rows.length > 0;
+ }

// Aplicar nas rotas GET /mensagens/:agendamentoId e POST /mensagens:
+ const temAcesso = await usuarioParticipaDoAgendamento(agendamentoId, req.user);
+ if (!temAcesso) return res.status(403).json({ ok: false, error: 'Acesso negado ao chat desta consulta.' });
```

Correção 4 (Dev 2): Proteção e Anti-Spoofing na Rota do Google Meet

```
// backend/src/server.js:136-168
- app.post('/api/meet/create-event', async (req, res) => {
+ app.post('/api/meet/create-event', requireAuth, async (req, res) => {
const { consultaId, summary, description, startDateTime, endDateTime } = req.body;
+ // Buscar participantes reais direto do banco de dados (impede spoofing de e-mails)
+ const ag = await query(`
+ SELECT a.id, up.email AS pac_email, ups.email AS psi_email FROM agendamentos a
+ JOIN pacientes pa ON pa.id = a.paciente_id JOIN usuarios up ON up.id = pa.usuario_id
+ JOIN psicologos ps ON ps.id = a.psicologo_id JOIN usuarios ups ON ups.id = ps.usuario_id
+ WHERE a.id = $1 AND (up.id = $2 OR ups.id = $2 OR $3 = 'admin')`,
+ [consultaId, req.user.id, req.user.tipo]);
+ if (ag.rows.length === 0) return res.status(403).json({ ok: false, error: 'Consulta não autorizada.' });
+ const pacienteEmail = ag.rows[0].pac_email;
+ const psicologoEmail = ag.rows[0].psi_email;
```

Correção 5 (Dev 3): Substituição de innerHTML por Manipulação Segura de DOM (Anti-XSS)

```
// frontend/js/script.js:247 (Renderização do chat):
function draw() {
const container = byId('chatMensagens');
container.innerHTML = '';
const mensagens = db().mensagens.filter(m => m.consultaId === id);
mensagens.forEach(m => {
const bubble = document.createElement('div');
bubble.className = `chat-bubble ${m.autorTipo === s.tipo ? 'mine' : ''}`;
const b = document.createElement('b'); b.textContent = m.autorTipo;
const p = document.createElement('p');
p.textContent = m.texto; // SEGURO: textContent não executa HTML/scripts!
bubble.appendChild(b); bubble.appendChild(p);
container.appendChild(bubble);
});
}
```

Correção 6 (Dev 4): Testes Automatizados de Regressão de Segurança

```
// backend/src/__tests__/database.test.js:
it('PATCH /api/agendamentos/:id/cancelar deve rejeitar IDOR com 403', async () => {
const res = await PATCH('/api/agendamentos/consulta-alheia-id/cancelar', { token: tokenInvasor });
assert.equal(res.status, 403);
});

it('GET /api/mensagens/:id deve rejeitar leitura não autorizada com 403', async () => {
const res = await GET('/api/mensagens/consulta-alheia-id', { token: tokenInvasor });
assert.equal(res.status, 403);
});

it('POST /api/meet/create-event deve exigir autenticação (401)', async () => {
const res = await POST('/api/meet/create-event', { body: { consultaId: '123' } });
assert.equal(res.status, 401);
});
```

4. Checklist de Validação para Code Review da Equipe

[ ] **JWT_SECRET:** Removido qualquer fallback estático; validação de tamanho mínimo (>=32 chars) ativa no boot.

[ ] **Variáveis de Ambiente:** Arquivo .env devidamente ignorado no .gitignore e sem chaves privadas commitadas.

[ ] **Controle de Propriedade (Anti-IDOR):** Cancelamento de agendamento restrito ao paciente, psicólogo ou administrador.

[ ] **Isolamento de Chat:** Validação obrigatória impedindo que usuários leiam ou postem em chats de consultas alheias.

[ ] **Google Meet:** Protegido por requireAuth com busca dos e-mails dos participantes direto do banco de dados.

[ ] **Proteção XSS:** Uso estrito de textContent / createElement no frontend para textos manipuláveis por usuários.

[ ] **Content Security Policy (CSP):** Reativada no middleware helmet com fontes estritas.

[ ] **Testes Automatizados:** Suíte de testes de regressão (npm test) executando com 100% de sucesso no backend.

[ ] **CI/CD:** Pipeline do GitHub Actions aprovado com todas as validações de segurança ativas.

Conclusão: Com a aplicação do plano de remediação deste guia, o sistema **PsicoConnect** atinge total conformidade com as diretrizes de desenvolvimento seguro (OWASP Top 10), garantindo o sigilo clínico exigido pelo Conselho Federal de Psicologia (CFP) e a conformidade legal com a Lei Geral de Proteção de Dados (LGPD).